Planeación de acciones usando un sistema basado en reglas para robots de servicio doméstico

Luis Sergio Cano Olguin

Tutor: Dr. Jesús Savage Carmona

Agradecimientos

Resumen

Índice general

1. Introducción **1**

1.1. Contexto y Motivación 2

1.2. Problemática: Sistemas Basados en Reglas en Entornos Dinámicos . . . 3

1.3. Hipótesis 3

1.4. Objetivos 4

1.5. Estructura del documento 5

2. Antecedentes y Estado del Arte **7**

2.1. Sistemas Basados en Reglas: Historia y Aplicaciones 7

2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones 8

2.2.1. Características Fundamentales de CLIPS 9

2.2.2. Ventajas de CLIPS en Robótica 10

2.2.3. Limitaciones de CLIPS en Entornos Dinámicos 10

2.3. Enfoques Modernos: Aprendizaje Automático (ML) y Redes Neuronales en Robótica 11

2.3.1. Principales Paradigmas de Aprendizaje Automático en Robótica 11

2.3.2. Aplicaciones Específicas en Robótica de Servicio 12

2.3.3. Limitaciones de los Enfoques Puramente Basados en ML 13

2.4. Sistemas Híbridos: Combinando la Predictibilidad de las Reglas con la Flexibilidad del ML 14

2.4.1. Definición y Fundamentos de los Sistemas Híbridos 14

2.4.2. Tipologías de Integración Híbrida 14

2.4.3. Ejemplos de Sistemas Híbridos en Robótica 15

2.4.4. Ventajas y Desafíos de los Sistemas Híbridos 16

2.4.5. Conexión con la Propuesta de Tesis 17

3. Marco Teórico y Conceptual 18
3.1. Arquitectura de un Sistema de Planeación de Acciones Jerárquico . . . 22
3.1.1. Fundamentos de la Planificación Jerárquica: HTN 22
3.1.2. El Planificador HATP (Hierarchical Agent-Based Task Planner) 23
3.1.3. Taxonomía de Arquitecturas Jerárquicas en Robótica 24
3.1.4. Relevancia para el Sistema Propuesto 25
3.2. Fundamentos de los Sistemas de Producción (Rule-Based Systems) . . 25
3.2.1. Estructura y Componentes de un Sistema de Producción 26
3.2.2. El Ciclo Reconocer-Actuar (Recognize-Act Cycle) 26
3.2.3. El Algoritmo RETE para Emparejamiento Eficiente de Patrones 27
3.2.4. Implementación de RETE en CLIPS 29
3.2.5. Ventajas y Limitaciones de los Sistemas de Producción 31
3.3. Introducción a los Modelos de Lenguaje Grande (LLMs) y ChatGPT/Qwen 32
3.3.1. Fundamentos Arquitectónicos: Transformers y Mecanismo de Atención 32
3.3.2. ChatGPT: Modelo Propietario para Interacción Conversacional 35
3.3.3. Qwen2.5-0.5B: Modelo Open-Source para Cómputo en el Borde 36
3.3.4. Comparativa y Selección para Robótica de Servicio 38
3.4. Integración de Traductores de Lenguaje Natural con Sistemas Basados en Reglas 40
3.4.1. Utilidad y Función de la Integración 40
3.4.2. Funcionamiento de la Integración CLIPS-Lenguaje Natural . . . 40
3.4.3. Descripción de Tecnologías: CLIPS, ChatGPT y Qwen2.5-0.5B . 40
3.5. ChatGPT como Agente de Planificación Autónomo 40
3.5.1. Capacidades de Planificación de LLMs vs. Sistemas Basados en Reglas 40
3.5.2. Arquitectura de Planificación Dual: CLIPS y ChatGPT como Motores Complementarios 40
3.5.3. Mecanismos de Selección y Validación de Planes 40
3.5.4. Ventajas y Riesgos de la Planificación con LLMs en Robótica . . 40

4. Metodología: Diseño del Sistema Híbrido CLIPS-ChatGPT/Qwen 41
4.1. Diseño del Sistema Basado en Reglas con CLIPS 41
4.1.1. Definición del Conjunto de Reglas Jerárquicas 41
4.1.2. Priorización de Acciones Críticas y Gestión de Emergencias . . . 41

4.1.3. Representación del Conocimiento y Hechos en la Base de Datos de CLIPS 41
4.2. Integración de ChatGPT/Qwen como Sistema Complementario 42
4.2.1. Funciones Asignadas al Módulo de Lenguaje Natural 42
4.2.2. Protocolo de Comunicación entre CLIPS y los Modelos de Lenguaje 42
4.2.3. Mecanismos de Seguridad y Validación para las Respuestas del LLM 42
4.3. Implementación de Planificación Dual 42
4.3.1. Diseño del Módulo de Planificación con ChatGPT 42
4.3.2. Protocolos de Prompting para Generación de Planes Robóticos . 42
4.3.3. Mecanismo de Selección entre Planes CLIPS vs. ChatGPT . . . 42
4.3.4. Validación y Simulación de Planes Antes de Ejecución 42

5. Implementación 44
5.1. Entornos de Desarrollo: Simulación y Plataforma Física 44
5.2. Implementación del Motor de Reglas en CLIPS 44
5.3. Desarrollo del Módulo de Integración 44
5.4. Configuración de la Interfaz con API de OpenAI para ChatGPT o Modelo Qwen Local 44
5.5. Casos de Prueba para Validar la Interacción entre los Módulos 44

6. Escenarios de Validación y Experimentación 45
6.1. Diseño de Experimentos en Entornos Controlados 45
6.1.1. Escenario 1: Ejecución de Tareas Predefinidas (solo CLIPS) . . 45
6.1.2. Escenario 2: Gestión de Órdenes Imprecisas o Novedosas (CLIPS + ChatGPT/Qwen) 45
6.1.3. Escenario 3: Respuesta a Eventos Inesperados o Fallos 45
6.1.4. Escenario 4: Planificación Dual para Tareas Complejas (CLIPS vs. ChatGPT) 45
6.2. Métricas de Evaluación 46
6.2.1. Métricas Cuantitativas: Tiempo de Ejecución, Tasa de Éxito, Uso de Recursos Computacionales 46
6.2.2. Métricas Cualitativas: Robustez, Interpretabilidad y Fluidez en la Interacción Humano-Robot 46

7. Análisis de Resultados **47**
7.1. Comparativa del Rendimiento del Sistema Solo-CLIPS vs. el Sistema Híbrido 47
7.2. Evaluación de la Efectividad de ChatGPT/Qwen en las Diferentes Funciones Asignadas 47
7.3. Discusión de Limitaciones y Errores 47
7.4. Análisis de la Escalabilidad y el Coste Computacional de la Integración 47

8. Discusión **48**
8.1. Interpretación de los Resultados en el Contexto de los Objetivos 48
8.2. Ventajas y Desventajas del Enfoque Híbrido Propuesto 48
8.3. Implicaciones para la Seguridad y Certificación en Entornos Regulados 48
8.4. Límites Éticos y Prácticos del Uso de ChatGPT/Qwen en Robótica . . 48

9. Contribuciones y Relevancia **49**
9.1. Contribución Técnica: Marco Híbrido Escalable y Documentado 49
9.2. Relevancia Práctica: Aplicaciones en Salud, Industria y Logística 49
9.3. Relevancia Académica: Benchmark para Sistemas Rule-Based vs. Híbridos 49

10. Conclusiones y Trabajo Futuro **50**
10.1. Conclusiones Principales 50
10.2. Propuestas de Trabajo Futuro 50

Índice de figuras

3.1. Arquitectura ViRoot. 19
3.2. Arquitectura Funcional ViRoot. 20

Capítulo 1

Introducción

Los robots de servicio doméstico representan un campo de proyección para la inteligencia artificial y la automatización, con aplicaciones que van desde la asistencia a personas mayores hasta la gestión autónoma de entornos residenciales (1). En este contexto, la planificación de acciones, es decir, la capacidad de un robot para descomponer objetivos de alto nivel en secuencias ejecutables, es un desafío en entornos dinámicos y no estructurados donde interactúan humanos, objetos móviles y tareas imprevistas (2).

Los sistemas basados en reglas, como los implementados en motores de inferencia del tipo CLIPS, han demostrado ser robustos y predecibles en escenarios estructurados, como líneas de producción o laboratorios controlados (3). Su fortaleza radica en la transparencia del razonamiento simbólico y la capacidad de priorizar acciones críticas, como la evasión de obstáculos o la gestión de emergencias. Sin embargo, su rigidez los hace poco adaptables ante variaciones no previstas en el entorno o ante comandos expresados en lenguaje natural con alto grado de ambigüedad o complejidad (4).

Recientemente, los modelos de lenguaje grande (LLMs), como ChatGPT, han emergido como herramientas capaces de interpretar y generar lenguaje natural, e incluso de actuar como agentes de planificación autónomos (5). Su flexibilidad contextual permite traducir instrucciones verbales en secuencias de acciones, complementando así la solidez de los sistemas basados en reglas. No obstante, su naturaleza probabilística y la falta de garantías formales de seguridad limitan su uso directo en aplicaciones robóticas donde la integridad física y la predictibilidad son prioritarias (6).

Esta tesis propone un **sistema híbrido de planificación de tareas** que integra un motor de reglas CLIPS con modelos de lenguaje natural del tipo ChatGPT, aprovechando las ventajas de ambos enfoques: la previsibilidad y seguridad de CLIPS, en

conjunto con la adaptabilidad y capacidad de comprensión lingüística de los LLMs. El sistema se implemento y valido en el robot de servicio doméstico *Justina*, desarrollado en el Laboratorio de Bio-Robótica de la UNAM, utilizando ROS 2 como marco de integración.

1.1. Contexto y Motivación

El área de la rebotica orientada al servicio doméstico ha evolucionado significativamente en la última década, impulsada por avances en percepción, planificación y interacción humano-robot (1). Estos sistemas se han diseñado para operar en entornos no estructurados como hogares, hospitales o centros de cuidado, donde deben realizar tareas que van desde la entrega de objetos hasta la asistencia en actividades de la vida diaria. En este contexto, la capacidad de planificar acciones de manera autónoma, segura y adaptativa se convierte en un requisito fundamental.

La planificación de acciones en robótica ha sido tradicionalmente abordada mediante sistemas basados en reglas *(Rule-Based Systems, RBS)*, que ofrecen un marco predecible y verificable para la toma de decisiones (2). Sin embargo, la creciente complejidad de los entornos domésticos—caracterizados por la presencia de humanos, objetos dinámicos y situaciones imprevistas—ha expuesto las limitaciones de los enfoques puramente simbólicos. Paralelamente, el reciente surgimiento de modelos de lenguaje grande **(LLMs)** ha abierto nuevas posibilidades para la interpretación de comandos en lenguaje natural y la generación de planes flexibles, aunque a menudo carentes de garantías formales de seguridad (6).

Esta tesis se desarrolla en el marco del Laboratorio de Bio-Robótica de la UNAM, utilizando como plataforma el robot de servicio Justina, un sistema modular basado en ROS 2 con capacidades avanzadas de navegación, manipulación y percepción. La motivación central del trabajo radica en la necesidad de desarrollar arquitecturas de planificación que combinen lo mejor de ambos mundos: la robustez y transparencia de los sistemas basados en reglas, y la adaptabilidad y capacidad de diálogo de los modelos de lenguaje modernos.

1.2. Problemática: Sistemas Basados en Reglas en Entornos Dinámicos

Los sistemas basados en reglas, implementados en motores de inferencia como CLIPS, se fundamentan en la lógica simbólica y el ciclo *reconocer-actuar* (3). Su principal ventaja es la previsibilidad: ante un conjunto de hechos y reglas bien definidos, la respuesta del sistema es determinista y explicable. Esto los hace idóneos para aplicaciones donde la seguridad y la certificación son críticas, como en entornos industriales o médicos.

No obstante, en contextos domésticos dinámicos, estos sistemas enfrentan desafíos significativos (4):

1. **Rigidez interpretativa:** No pueden procesar comandos en lenguaje natural complejo o ambiguo, como "trae el libro rojo que está cerca de la ventana".
2. **Falta de adaptabilidad:** Las reglas deben ser definidas a priori; cualquier situación no prevista en la base de conocimiento puede llevar al fracaso o a un comportamiento no deseado.
3. **Dificultad para gestionar la incertidumbre:** No manejan bien la información incompleta o cambiante del entorno en tiempo real.
4. **Escalabilidad limitada:** Mantener y extender un conjunto grande de reglas para cubrir todos los escenarios posibles resulta complejo y laborioso.

Por otro lado, los enfoques puramente basados en aprendizaje automático — especialmente los LLMs—ofrecen flexibilidad y capacidad de generalización, pero introducen riesgos como (6):

- Comportamientos impredecibles o no verificables.
- Falta de garantías de seguridad en la ejecución física.
- Dependencia de grandes volúmenes de datos y recursos computacionales.

La problemática central, por tanto, es cómo diseñar un sistema de planificación que mantenga la seguridad y explicabilidad de los RBS, pero que sea lo suficientemente flexible para operar en entornos domésticos dinámicos e interactuar de manera natural con humanos.

1.3. Hipótesis

La hipótesis central de esta investigación propone la integración sinérgica de un sistema basado en reglas (CLIPS) con un modelo de lenguaje grande (ChatGPT o

Qwen) puede superar las limitaciones de cada enfoque por separado, resultando en un sistema híbrido de planificación que es a la vez seguro, explicable, adaptable y capaz de entender lenguaje natural.

Esta complementariedad se articula en tres niveles:

1. **Nivel de interpretación:** Los LLMs actúan como traductores de lenguaje natural a hechos estructurados, permitiendo que comandos complejos y ambiguos sean convertidos en representaciones simbólicas que CLIPS puede procesar.
2. **Nivel de planificación:** Se implementa una arquitectura de planificación dual, donde CLIPS genera planes predecibles y seguros para tareas conocidas, mientras que los LLMs proponen soluciones flexibles para situaciones novedosas o complejas.
3. **Nivel de validación y selección:** Un mecanismo de supervisión evalúa los planes generados por ambos motores con base en criterios de seguridad, eficiencia y contexto, seleccionando la mejor opción o combinándolas de manera segura.

Se espera que este sistema híbrido:

- Mejore la tasa de éxito en la ejecución de tareas en entornos domésticos.
- Reduzca el tiempo de respuesta ante comandos complejos.
- Mantenga un comportamiento seguro y predecible en situaciones críticas.
- Permita una interacción más natural e intuitiva con usuarios no expertos.

La validación de esta hipótesis se realizará mediante experimentos en simulación y con el robot físico Justina, comparando el desempeño del sistema híbrido frente a enfoques puramente basados en reglas o puramente basados en LLMs.

1.4. Objetivos

- **Diseño del Sistema Híbrido:**
  - Definir un conjunto jerárquico de reglas en CLIPS para priorizar acciones críticas.
  - Especificar la arquitectura de integración entre CLIPS y modelos de lenguaje natural.
  - Diseñar un marco de planificación dual que permita tanto a CLIPS como a ChatGPT generar y ejecutar planes.
  - Establecer protocolos de comunicación y validación entre los módulos.
- **Implementación Técnica:**

    - Integrar el motor de reglas CLIPS con ROS (Robot Operating System).
    - Desarrollar el módulo de interfaz con APIs de ChatGPT o implementación local de Qwen2.5-0.5B.
    - Implementar capacidades de planificación autónoma en ChatGPT mediante prompting estructurado.
    - Desarrollar un mecanismo de selección y validación entre planes generados por diferentes motores.
- **Capacidades de Interacción Natural:**
    - Implementar traducción de comandos en lenguaje natural a hechos estructurados de CLIPS.
    - Desarrollar mecanismos de diálogo para resolución de ambigüedades.
    - Crear protocolos de retroalimentación para aprendizaje incremental.
- **Validación Experimental:**
    - Validar el sistema en escenarios realistas con el robot físico y simulaciones.
    - Evaluar el rendimiento mediante métricas comparativas:
        - Tiempo de ejecución de tareas y tasa de éxito.
        - Precisión en interpretación de comandos complejos.
        - Robustez ante instrucciones ambiguas o novedosas.
    - Comparar el sistema híbrido vs. abordajes puramente basados en reglas.
    - Evaluar la calidad de planes generados por CLIPS vs. ChatGPT en diferentes escenarios.

1.5. Estructura del documento

Este documento está organizado en los siguientes capítulos que describen el sistema propuesto para el desarrollo de esta tesis, abarcando el planteamiento, desarrollo, implementación y validación del sistema híbrido de planificación. La estructura es la siguiente:

1. **Capítulo 1: Introducción.** Presenta el contexto, motivación, problemática, hipótesis y objetivos de la investigación, así como la justificación del enfoque híbrido en robótica de servicio doméstico.
2. **Capítulo 2: Antecedentes y estado del arte.** Revisa la evolución de los sistemas basados en reglas en robótica, las características de CLIPS, los enfoques modernos de aprendizaje automático y los sistemas híbridos existentes.

3. **Capítulo 3: Marco teórico y conceptual.** Describe los fundamentos de la planificación jerárquica, los sistemas de producción, los modelos de lenguaje grande (LLMs) y la arquitectura de integración propuesta.
4. **Capítulo 4: Metodología.** Detalla el diseño del sistema híbrido, incluyendo la definición de reglas en CLIPS, la integración con ChatGPT/Qwen, y los mecanismos de planificación dual y validación.
5. **Capítulo 5: Implementación.** Explica los entornos de desarrollo, la configuración técnica, la interfaz con APIs y las pruebas utilizados para validar la integración de los módulos.
6. **Capítulo 6: Escenarios de validación y experimentación.** Presenta el diseño experimental, los escenarios de prueba y las métricas cuantitativas y cualitativas utilizadas para evaluar el sistema.
7. **Capítulo 7: Análisis de resultados.** Compara el rendimiento del sistema híbrido frente al sistema basado solo en reglas, evalúa la efectividad de los LLMs y discute limitaciones y costos computacionales.
8. **Capítulo 8: Discusión.** Interpreta los resultados en relación con los objetivos, analiza ventajas y desventajas del enfoque híbrido, y explora implicaciones éticas y de seguridad.
9. **Capítulo 9: Contribuciones y relevancia.** Destaca la aportación técnica del marco híbrido, su aplicabilidad práctica en distintos sectores y su relevancia académica como benchmark.
10. **Capítulo 10: Conclusiones y trabajo futuro.** Resume los hallazgos principales y propone líneas de investigación futura, como el fine-tuning de LLMs y la mejora de mecanismos de seguridad.

Capítulo 2

Antecedentes y Estado del Arte

Se realiza una revisión comprensiva de la evolución histórica de los sistemas basados en reglas en el área de la robótica, analizando aplicaciones tradicionales y limitaciones actuales. También se examinan las características técnicas de CLIPS y se contrastan con enfoques modernos de aprendizaje automático, culminando con el estudio de sistemas híbridos que combinan la predictibilidad de las reglas con la flexibilidad del *(Machine Learning, ML)* para entornos dinámicos.

2.1. Sistemas Basados en Reglas: Historia y Aplicaciones

Los sistemas basados en reglas *(Rule-Based Systems, RBS)*, constituyen uno de los paradigmas más antiguos y consolidados en inteligencia artificial (3). Su origen se remonta a los sistemas expertos de la década de 1970, donde se utilizaban para representar el conocimiento de especialistas humanos en dominios bien delimitados, como el diagnóstico médico o la configuración de sistemas complejos (2). En robótica, la adopción de estos sistemas se popularizó en los años 80 y 90, particularmente en entornos industriales estructurados, donde la previsibilidad y seguridad eran prioritarias.

Históricamente, los RBS han sido fundamentales en arquitecturas de control robótico jerárquico, como la propuesta por Brooks (4), donde capas de comportamientos reactivos podían ser coordinadas mediante reglas de supresión. Más adelante, motores de inferencia como CLIPS *(C Language Integrated Production System)*, desarrollado por la NASA en 1985, se convirtieron en herramientas estándar para la implementa-

ción de sistemas de planificación y toma de decisiones en robots autónomos (7). CLIPS permitió la representación simbólica del conocimiento a través de hechos, reglas, en conjunto con un ciclo de inferencia *reconocer-actuar* que ofreció un marco eficiente para el razonamiento en tiempo real.

En cuanto a aplicaciones, los sistemas basados en reglas han demostrado su utilidad en diversos dominios robóticos:

1. **Robótica industrial:** En cadenas de montaje automatizadas, donde las tareas son repetitivas y el entorno está controlado. Los RBS se utilizan para secuenciar operaciones, gestionar excepciones y garantizar la seguridad de los operarios humanos (1).
2. **Robótica de servicio y doméstica:** En tareas simples como la entrega de objetos, navegación en interiores y asistencia a personas con movilidad reducida.
3. **Robótica espacial y de exploración:** Sistemas como Remote Agent de la NASA emplearon arquitecturas basadas en reglas para la planificación y ejecución autónoma de experimentos en misiones no tripuladas (8).
4. **Sistemas híbridos contemporáneos:** Recientemente, los RBS se han integrado con técnicas de aprendizaje automático para compensar sus limitaciones en entornos dinámicos. Por ejemplo, en *Robotic Pick-and-Place with Natural Language Commands* (9) se combina un RBS con un modelo de lenguaje natural para interpretar comandos verbales en tareas de manipulación.

A pesar de la relevancia de enfoques basados en aprendizaje profundo, los sistemas basados en reglas mantienen su relevancia en aplicaciones donde la explicabilidad, verificabilidad y seguridad son críticas (6). Su capacidad para representar conocimiento simbólico de forma transparente los hace ideales para entornos regulados o donde se requiere auditoría de las decisiones del robot.

2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones

CLIPS es un lenguaje de programación basado en reglas, diseñado específicamente para la construcción de sistemas expertos y aplicaciones basadas en conocimiento (7). Concebido originalmente como una alternativa a los costosos sistemas implementados en LISP y en estaciones de trabajo especializadas, CLIPS destacó por su portabilidad, eficiencia y su capacidad para ejecutarse en hardware de bajo costo (3). Su adopción se

extendió rápidamente en la comunidad de robótica e inteligencia artificial, convirtiéndose en una herramienta de referencia para la implementación de sistemas de producción (2).

2.2.1. Características Fundamentales de CLIPS

CLIPS se fundamenta en el paradigma de los sistemas de producción, operando mediante su ciclo de inferencia que permite razonar sobre una base de hechos utilizando reglas del tipo if-then (3). Entre sus características más distintivas se encuentran:

1. **Encadenamiento hacia adelante *(forward chaining)*:** CLIPS utiliza predominantemente encadenamiento hacia adelante, lo que significa que, partiendo de un conjunto de hechos iniciales, el motor de inferencia aplica reglas para derivar nuevos hechos de manera iterativa (7). Este enfoque es especialmente adecuado para aplicaciones en tiempo real donde los cambios en el entorno deben reflejarse inmediatamente en la base de conocimiento (1).
2. **Algoritmo RETE para emparejamiento de patrones:** CLIPS implementa una versión optimizada del algoritmo RETE, que permite un emparejamiento eficiente de patrones entre reglas y hechos, incluso cuando la base de conocimiento contiene miles de elementos (3). Esta característica es crítica en robótica, donde la velocidad de respuesta es determinante (4).
3. **Representación del conocimiento multifacética:** Además de reglas, CLIPS permite representar conocimiento mediante hechos ordenados y plantillas *(deftemplate)*, así como mediante funciones definidas por el usuario en C (7). Esto facilita la integración con sistemas de percepción y control de bajo nivel (10).
4. **Portabilidad y código abierto:** Escrito en C, CLIPS puede ejecutarse en prácticamente cualquier plataforma, desde microcontroladores hasta sistemas de alto rendimiento (3). Su naturaleza open source ha permitido su adaptación en múltiples contextos robóticos, como el control de robots de exploración y sistemas de inspección (7).
5. **Integración con otros lenguajes:** CLIPS ofrece una API en C y una librería en python, lo que le permite ser empleado en aplicaciones más grandes, posibilitado su integración con frameworks robóticos como ROS y entornos de simulación (1).

2.2.2. Ventajas de CLIPS en Robótica

La adopción de CLIPS en robótica de servicio y sistemas autónomos se justifica por varias ventajas clave:

- **Explicabilidad y transparencia:** Las decisiones tomadas por un sistema CLIPS pueden ser rastreadas y explicadas mediante la inspección de las reglas activadas y los hechos utilizados (3). Esto es fundamental en aplicaciones donde la seguridad y la auditoría son prioritarias, como en robótica médica o espacial (6).
- **Previsibilidad del comportamiento:** Dado que el conjunto de reglas es definido explícitamente, el comportamiento del sistema es determinista y verificable formalmente (2). Esta característica contrasta con los enfoques de deep learning, donde las decisiones pueden ser opacas y difíciles de validar (6).
- **Bajo consumo de recursos computacionales:** CLIPS puede ejecutarse en hardware con capacidades limitadas, como robots móviles embarcados, sin necesidad de GPUs o grandes cantidades de memoria (1). Esto lo hace adecuado para plataformas robóticas con restricciones energéticas (10).
- **Facilidad de mantenimiento y actualización:** La base de conocimiento puede ser modificada o extendida sin necesidad de recompilar el sistema completo, lo que facilita la adaptación a nuevos entornos o requisitos (3).

2.2.3. Limitaciones de CLIPS en Entornos Dinámicos

A pesar de sus fortalezas, CLIPS presenta limitaciones significativas cuando se aplica en entornos domésticos dinámicos y no estructurados:

1. **Rigidez semántica:** CLIPS no puede interpretar directamente comandos en lenguaje natural complejo o ambiguo; requiere que la entrada sea previamente estructurada en hechos formales (9). Esto limita su capacidad para interactuar con usuarios no expertos de manera intuitiva (6).
2. **Falta de aprendizaje automático:** CLIPS es un sistema simbólico puro; no posee mecanismos intrínsecos para aprender de la experiencia o adaptar sus reglas automáticamente ante situaciones novedosas (2). Cualquier modificación debe ser realizada manualmente por un programador.
3. **Escalabilidad del conocimiento:** Mantener una base de reglas que cubra todas las posibles situaciones en un hogar resulta complejo y propenso a inconsistencias (4). A medida que crece el número de reglas, la gestión y depuración se vuelven

más difíciles (3).

4. **Manejo limitado de la incertidumbre:** CLIPS no incorpora mecanismos nativos para representar información incierta o probabilística, lo que dificulta su uso en tareas de percepción donde los sensores proporcionan datos ruidosos (1).
5. **Integración compleja con subsistemas modernos:** Aunque existen iniciativas como el **ROS 2 CLIPS-Executive** (10), la integración de CLIPS con pipelines de percepción basados en aprendizaje profundo requiere desarrollos ad hoc que no siempre son triviales (9).

   Estas limitaciones han motivado la exploración de arquitecturas híbridas que combinen la robustez y transparencia de CLIPS con la flexibilidad y capacidad de adaptación de los modelos de lenguaje grande, como se discutirá en secciones posteriores de esta tesis (6).

2.3. Enfoques Modernos: Aprendizaje Automático (ML) y Redes Neuronales en Robótica

El desarrollo del aprendizaje automático, y en particular del aprendizaje profundo, ha transformado radicalmente la robótica contemporánea, habilitando capacidades que eran difíciles de lograr con enfoques puramente simbólicos o geométricos (11). A diferencia de los sistemas basados en reglas, que requieren la especificación explícita de todo el conocimiento del dominio, los enfoques de ML permiten a los robots aprender directamente de los datos, extrayendo patrones complejos y adaptándose a entornos variables sin necesidad de programación explícita (2).

2.3.1. Principales Paradigmas de Aprendizaje Automático en Robótica

Aprendizaje Supervisado y Visión por Computadora: Las redes neuronales convolucionales (CNNs) se han convertido en el estándar de facto para tareas de percepción robótica, incluyendo detección y reconocimiento de objetos, segmentación semántica y estimación de poses (12). Arquitecturas como YOLO (You Only Look Once) y sus variantes permiten a robots como Justina identificar objetos en tiempo real con alta precisión, incluso en entornos domésticos desordenados . Las CNNs operan mediante capas de convolución que extraen jerárquicamente características visuales,

desde bordes simples hasta formas complejas, permitiendo una representación robusta del entorno (12).

Aprendizaje por Refuerzo para Control y Navegación: El aprendizaje por refuerzo profundo (Deep RL) ha emergido como una herramienta poderosa para desarrollar políticas de control en tareas secuenciales. Mediante la interacción con el entorno, los agentes aprenden a maximizar recompensas acumulativas, descubriendo estrategias óptimas para navegación, manipulación y evitación de obstáculos (13). En navegación robótica, se han desarrollado políticas híbridas que combinan representaciones aprendidas con modelos geométricos tradicionales para operar robustamente en entornos reales (13).

Redes Recurrentes y Modelado Secuencial: Las redes neuronales recurrentes (RNNs) y sus variantes (LSTM, GRU) son fundamentales para tareas que requieren memoria temporal, como el seguimiento de objetos en movimiento, la predicción de trayectorias humanas o la interpretación de comandos verbales en contexto (11). Estas arquitecturas mantienen un estado interno que evoluciona con la secuencia de entradas, permitiendo modelar dependencias temporales de largo alcance.

Modelos de Lenguaje Grande (LLMs) en Robótica: La irrupción de modelos transformadores, como los descritos por (5), han dado lugar a los Large Language Models (LLMs), capaces de procesar y generar lenguaje natural con una fluidez sin precedentes. En robótica, estos modelos se utilizan para interpretar comandos complejos, traducir lenguaje natural a representaciones simbólicas, e incluso generar planes completos a partir de descripciones de alto nivel (6). Su capacidad para captar matices semánticos y contextuales los hace especialmente valiosos para la interacción humano-robot en entornos domésticos.

2.3.2. Aplicaciones Específicas en Robótica de Servicio

- **Percepción y Reconocimiento:** La integración de CNNs en pipelines de percepción permite a los robots identificar objetos, personas y gestos con alta precisión. Sistemas como YOLO y FastSAM (Segment Anything Model) se utilizan para segmentación en tiempo real y localización de objetos de interés (12).
- **Navegación Autónoma:** Los enfoques basados en aprendizaje profundo han demostrado superioridad frente a métodos geométricos tradicionales en entornos desconocidos, utilizando entradas como mapas locales de obstáculos (LOM) para generar comandos de velocidad directamente (13). La combinación de SLAM vi-

sual con redes neuronales permite una localización robusta incluso en escenarios dinámicos (11).

- **Manipulación y Control:** En robótica de manipulación, las redes neuronales se emplean para estimar propiedades físicas de objetos (rigidez, forma) a partir de señales propioceptivas, permitiendo agarres adaptativos y seguros (14).
- **Planificación y Toma de Decisiones:** Los algoritmos de planificación de rutas y decisiones en vehículos autónomos combinan enfoques basados en conocimiento (reglas, teoría de juegos) con métodos basados en datos (imitación, RL) para lograr sistemas robustos y adaptativos (**?**).

2.3.3. Limitaciones de los Enfoques Puramente Basados en ML

A pesar de sus ventajas, los enfoques puramente basados en ML presentan limitaciones significativas:

1. **Falta de garantías de seguridad:** Las redes neuronales pueden producir comportamientos impredecibles ante entradas fuera de la distribución de entrenamiento, lo que es crítico en aplicaciones donde la seguridad física es prioritaria (6).
2. **Opacidad y falta de explicabilidad:** La naturaleza de *çaja negra"* de las redes profundas dificulta la interpretación de sus decisiones, limitando su adopción en entornos regulados donde se requiere auditoría (2).
3. **Dependencia de datos:** Los modelos de ML requieren grandes volúmenes de datos etiquetados o experiencias de interacción, cuyo costo de adquisición en robótica suele ser complejo (11).
4. **Dificultad para incorporar conocimiento previo:** A diferencia de los sistemas basados en reglas, las redes neuronales no pueden aprovechar fácilmente el conocimiento simbólico existente sobre el dominio (4).

Estas limitaciones han motivado el desarrollo de arquitecturas híbridas que combinan lo mejor de ambos paradigmas.

2.4. Sistemas Híbridos: Combinando la Predictibilidad de las Reglas con la Flexibilidad del ML

Los sistemas híbridos en robótica representan una tercera vía que busca integrar sinérgicamente enfoques simbólicos (basados en reglas) y subsimbólicos (basados en aprendizaje), aprovechando las fortalezas de cada paradigma mientras se compensan sus debilidades respectivas (11).

2.4.1. Definición y Fundamentos de los Sistemas Híbridos

Un sistema híbrido de planificación y control esta definido como aquella arquitectura que integra explícitamente componentes basados en reglas (deterministas, explicables, verificables) con componentes basados en aprendizaje (adaptativos, flexibles, basados en datos) (13). El objetivo no es sustituir un enfoque por otro, sino establecer mecanismos de coordinación que permitan a cada módulo operar en el ámbito donde es más competente.

La motivación fundamental de los sistemas híbridos radica en la complementariedad de ambos paradigmas (**?**):

- Los sistemas basados en reglas aportan predictibilidad, seguridad, explicabilidad y la capacidad de incorporar conocimiento experto explícito.
- Los enfoques basados en ML aportan adaptabilidad, capacidad de generalización a situaciones novedosas, y la habilidad de procesar entradas complejas y ruidosas (como imágenes o lenguaje natural).

2.4.2. Tipologías de Integración Híbrida

En la literatura se identifican varios patrones arquitectónicos para la integración híbrida (**?**):

1. **Arquitecturas de Dominio de Reglas con Complemento ML (Rule First + ML):** En este enfoque, el sistema basado en reglas constituye el núcleo principal de toma de decisiones, y los componentes de ML se utilizan para extender sus capacidades en áreas específicas. Por ejemplo, un sistema CLIPS puede manejar la lógica de alto nivel de una tarea, mientras que una CNN se encarga de la detección de objetos necesaria para instanciar los hechos.

2. **Arquitecturas de Dominio ML con Restricciones de Reglas (Model First + Rules):** Aquí, un modelo de ML (como una política de RL o un LLM) genera las acciones principales, pero sus salidas son validadas y, si es necesario, corregidas por un conjunto de reglas de seguridad que actúan como *çaja de seguridad"* (6). Esto es común en navegación autónoma, donde el planificador basado en aprendizaje puede sugerir trayectorias, pero un módulo de reglas garantiza que se eviten obstáculos y se respeten límites de velocidad.
3. **Arquitecturas de Fusión o Scheduling Paralelo:** En este esquema, tanto el sistema basado en reglas como el modelo de ML generan acciones concurrentemente, y un módulo supervisor (scheduler) decide qué acción ejecutar basándose en el contexto, la confianza de cada módulo o criterios predefinidos (13). Este enfoque permite aprovechar la redundancia y seleccionar la mejor estrategia para cada situación.
4. **Arquitecturas de Aprendizaje para Adaptación de Reglas:** En sistemas más avanzados, los componentes de ML se utilizan para adaptar o ajustar dinámicamente los parámetros del sistema basado en reglas. Por ejemplo, un meta-aprendizaje basado en Soft Actor-Critic (SAC) puede ajustar en línea los factores de escala de un controlador PID difuso, mejorando el rendimiento sin perder la estabilidad del lazo determinista de alto frecuencia (15).

2.4.3. Ejemplos de Sistemas Híbridos en Robótica

- **Navegación Híbrida:** La tesis de (13) propone agentes autónomos que combinan técnicas geométricas tradicionales (modelado explícito del entorno) con políticas aprendidas mediante redes neuronales, demostrando que los enfoques híbridos pueden operar robustamente tanto en simulación como en entornos físicos reales.
- **Análisis Semántico para Interacción Humano-Robot:** Se han desarrollado enfoques híbridos de análisis semántico que combinan técnicas basadas en reglas con aprendizaje automático para interpretar comandos de lenguaje natural en asistentes robóticos, logrando una comprensión más robusta y contextual (16).
- **Control de Robots Industriales:** La combinación de controladores PID difusos (basados en 49 reglas) con meta-optimización mediante aprendizaje por refuerzo (SAC) ha demostrado reducciones significativas en error de seguimiento ($46 - 52\,\%$) y consumo energético ($28 - 30\,\%$) en manipuladores industriales, manteniendo la estabilidad y predictibilidad del lazo de control de alta frecuencia

(15).

- **Planificación Híbrida (Hybrid Planning):** El paradigma de Hybrid Planning integra explícitamente sistemas de reglas (árboles de tareas, máquinas de estado finito) con módulos de predicción basados en modelos (MPC, LLMs) para lograr comportamientos robustos en entornos complejos citeHybridPlanning2024. Este enfoque se aplica en robótica de almacenes, interacción humano-robot y vehículos autónomos.
- **Modelado Físico con Aprendizaje:** En robótica de robots blandos y paralelos, se están explorando enfoques que combinan modelos físicos con aprendizaje automático (hybrid learning physics modeling) para mejorar la precisión del modelado cinemático y dinámico, aprovechando el conocimiento previo de la estructura del sistema mientras se aprenden compensaciones de datos (14).

2.4.4. Ventajas y Desafíos de los Sistemas Híbridos

Ventajas:

(6, 13)

- **Robustez mejorada:** Al combinar múltiples fuentes de conocimiento, el sistema puede fallar de manera controlada y recurrir a estrategias alternativas.
- **Seguridad preservada:** Las reglas pueden actuar como cortafuegos que previenen comportamientos inseguros generados por modelos de ML.
- **Explicabilidad selectiva:** Las decisiones críticas pueden ser explicadas mediante las reglas que las generaron, mientras que las decisiones rutinarias pueden delegarse a modelos opacos.
- **Adaptabilidad sin sacrificar estabilidad:** Los componentes de ML permiten adaptación a contextos cambiantes, mientras que las reglas mantienen un comportamiento base estable y verificado.

Desafíos:

(11)

- **Complejidad de integración:** La coordinación entre módulos heterogéneos requiere diseños arquitectónicos cuidadosos y protocolos de comunicación eficientes.
- **Resolución de conflictos:** Es necesario definir mecanismos claros para cuando las recomendaciones de reglas y modelos entren en conflicto.

- **Validación y verificación:** Los sistemas híbridos heredan la dificultad de verificar componentes de ML, aunque la presencia de reglas puede facilitar la validación del comportamiento global.
- **Sobrecarga computacional:** Ejecutar múltiples motores de razonamiento concurrentemente puede aumentar los requisitos de cómputo a bordo.

2.4.5. Conexión con la Propuesta de Tesis

La arquitectura híbrida CLIPS-ChatGPT propuesta en esta tesis se enmarca precisamente en esta categoría de sistemas, siguiendo un patrón de dominio de reglas con complemento ML para la traducción de lenguaje natural (10),, y extendiéndose hacia una arquitectura de planificación dual que permite tanto a CLIPS como a los LLMs generar planes concurrentemente (6). Esta aproximación busca capitalizar la predictibilidad y seguridad de CLIPS para tareas críticas y bien definidas, mientras que aprovecha la flexibilidad semántica y capacidad de generación contextual de ChatGPT para interpretar comandos complejos y adaptarse a situaciones novedosas. La validación experimental de esta hipótesis de complementariedad constituye el núcleo de la contribución de esta investigación.

Capítulo 3

Marco Teórico y Conceptual

El fundamento teórico del sistema propuesto consiste en una arquitectura de planeación jerárquica. Se introduce el funcionamiento de los modelos de lenguaje grande y se detalla la integración de traductores de lenguaje natural con sistemas basados en reglas, estableciendo las bases tecnológicas para la implementación del sistema híbrido CLIPS-ChatGPT/Qwen y su uso en la arquitectura ViRoot *(Virtual Real Robot)* descrita a continuación (ver Figura 3.1).

Arquitectura del Sistema

La arquitectura VIRBOT de Justina se organiza en cuatro capas principales:

- **Capa de Entradas**
  - Procesamiento de datos de sensores internos y externos.
  - Aplicación de técnicas de reconocimiento de patrones.
  - Generación del estado del entorno.
- **Capa de Planificación**
  - Validación mediante gestión del conocimiento.
  - Reconocimiento de situaciones y activación de objetivos.
  - Planificación de secuencias de operaciones físicas.
- **Capa de Gestión del Conocimiento**
  - Mapas del entorno creados con técnicas SLAM.
  - Sistema de localización basado en filtro de Kalman.
  - Sistema basado en reglas CLIPS para representación del conocimiento.
- **Capa de Ejecución**
  - Ejecución y verificación de planes de movimiento.

- Procedimientos predefinidos representados como máquinas de estado.
- Integración de procedimientos para generar planes complejos.

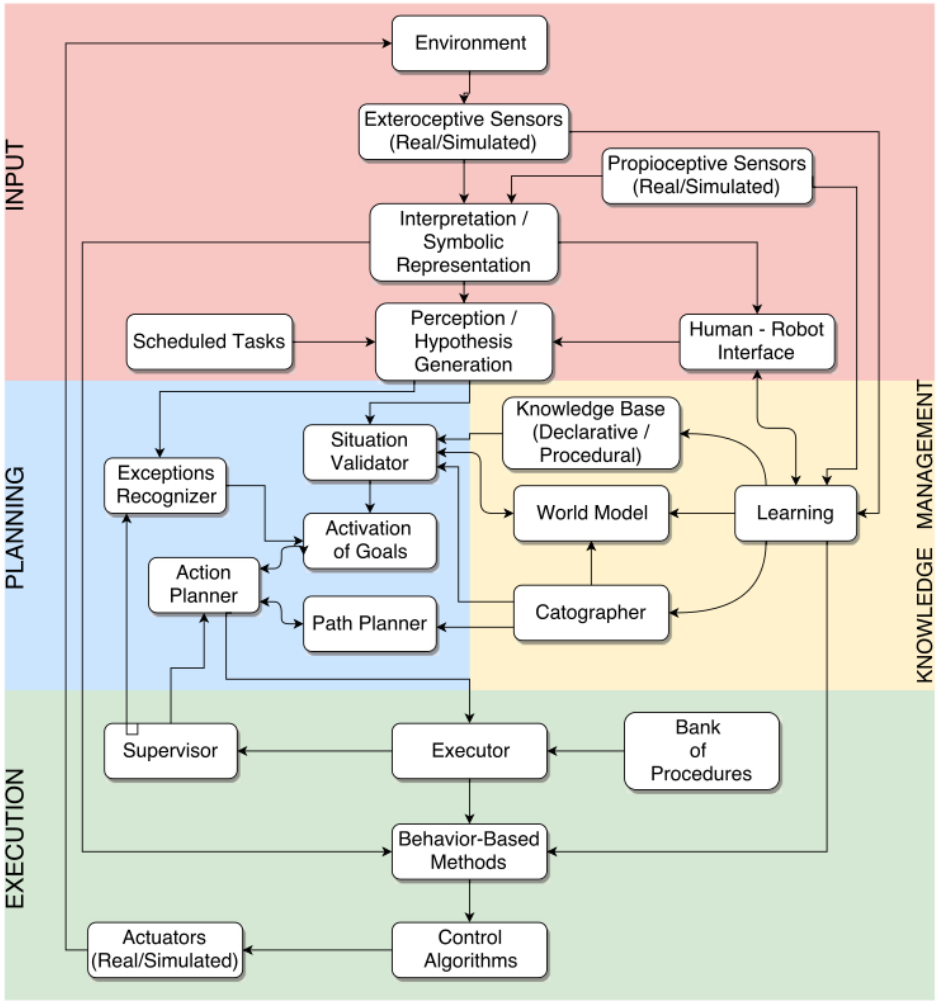

Figura 3.1: Arquitectura ViRoot.

Arquitectura Funcional de ViRbot

Esta arquitectura organiza los módulos funcionales del robot de manera interconectada, lo que permiten integrar la percepción, comprensión del lenguaje, planeación simbólica y control físico del robot. La Figura 3.2 muestra la interacción entre los distintos subsistemas del robot Justina.

Dentro de la arquitectura se distinguen los cuatro grupos principales: percepción, comprensión e interacción, planificación y supervisión, y control de hardware.

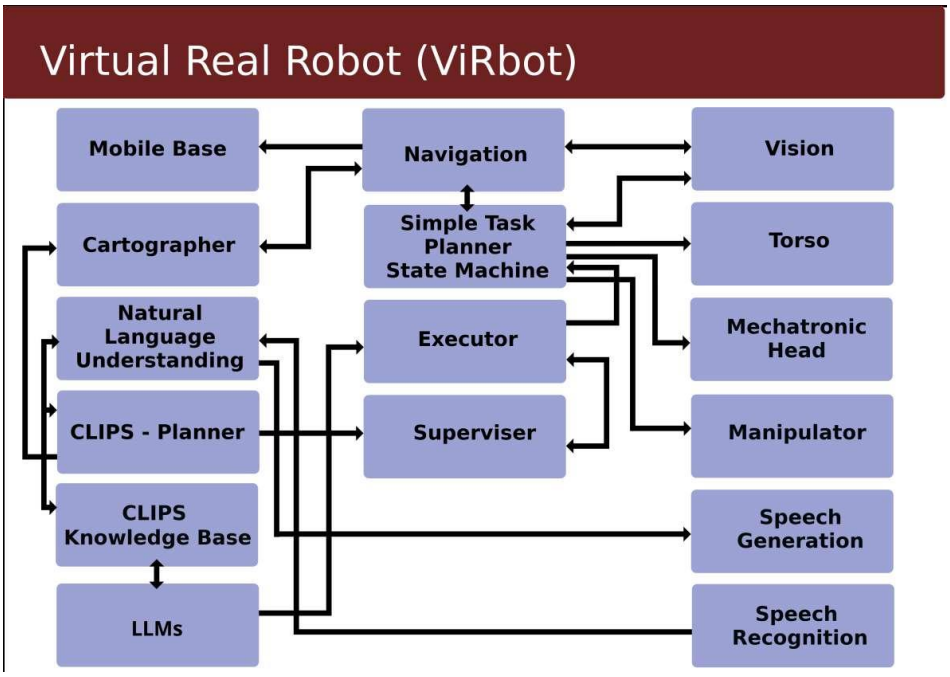

Figura 3.2: Arquitectura Funcional ViRoot.

Módulos de Percepción

Los módulos de percepción permiten al robot obtener información del entorno físico mediante sensores y algoritmos de interpretación.

- **Vision:** Este módulo procesa información proveniente de cámaras y sensores visuales *(nubes de puntos)*. Permite la detección y reconocimiento de objetos, personas y características del entorno. La información visual es utilizada principalmente por el sistema de navegación y por el planificador de tareas para tomar decisiones sobre la interacción con el entorno.
- **Cartographer:** El módulo Cartographer implementa algoritmos de **SLAM** *(Simultaneous Localization and Mapping)* para construir mapas del entorno y estimar la posición del robot dentro de ellos. Este módulo proporciona información fundamental para la navegación autónoma.
- **Speech Recognition:** Este módulo procesa las señales de audio capturadas por los micrófonos del robot y las convierte en texto. El resultado es utilizado por el sistema de *Natural Language Understanding* para interpretar instrucciones del usuario.

Módulos de Comprensión e Interacción

Estos módulos permiten que el robot interactúe con los humanos mediante lenguaje natural.

- **Natural Language Understanding (NLU):** Este componente interpreta las instrucciones del usuario en lenguaje natural. Utiliza modelos lingüísticos y reglas semánticas para transformar frases en representaciones estructuradas que puedan ser utilizadas por el sistema de planificación.
- **LLMs:** Los Large Language Models (LLMs) se emplean como complemento para mejorar la interpretación semántica de las instrucciones. Estos modelos permiten generar representaciones simbólicas o estructuras intermedias que posteriormente son procesadas por el sistema de planificación basado en reglas.
- **Speech Generation:** Este módulo permite al robot generar respuestas habladas. Convierte texto generado por el sistema en señales de voz, facilitando la interacción natural con los usuarios.

Módulos de Planificación y Gestión del Conocimiento

Estos módulos constituyen el núcleo de la toma de decisiones del sistema.

- **CLIPS Knowledge Base:** La base de conocimiento implementada en CLIPS almacena hechos y reglas que representan el conocimiento del entorno, los objetos, las tareas y las capacidades del robot. Esta base de conocimiento se actualiza continuamente conforme el robot percibe el entorno.
- **CLIPS Planner:** Este módulo utiliza reglas de producción para generar planes de acción a partir de objetivos definidos. El planificador determina qué acciones deben ejecutarse y en qué orden para alcanzar una meta específica.
- **Simple Task Planner State Machine:** Este componente implementa una máquina de estados que coordina la ejecución de tareas simples. Funciona como un nivel intermedio entre la planificación simbólica y la ejecución de acciones físicas.
- **Supervisor:** El supervisor monitorea la ejecución de los planes y gestiona errores o situaciones inesperadas. También se encarga de coordinar la interacción entre los distintos módulos del sistema.
- **Executor:** El ejecutor recibe los planes generados por el planificador y los convierte en comandos concretos para los diferentes actuadores del robot.

Módulos de Control del Robot

Estos módulos controlan directamente los sistemas físicos del robot.

- **Navigation:** El sistema de navegación calcula trayectorias y controla el desplazamiento del robot utilizando información del mapa y de los sensores.
- **Mobile Base:** Este módulo controla la base móvil del robot, permitiendo su desplazamiento en el entorno.
- **Manipulator:** Controla el brazo robótico utilizado para manipular objetos en el entorno.
- **Torso:** Permite el movimiento vertical del cuerpo del robot para alcanzar objetos a diferentes alturas.
- **Mechatronic Head:** Controla los movimientos de la cabeza del robot, permitiendo orientar sensores como cámaras y micrófonos hacia puntos de interés.

3.1. Arquitectura de un Sistema de Planeación de Acciones Jerárquico

La planificación de acciones aborda el problema de generar secuencias de pasos u operaciones que permitan a un robot alcanzar objetivos de alto nivel en entornos parcialmente observables y dinámicos (2). Entre los paradigmas más consolidados para abordar este desafío se encuentra la planificación jerárquica, que descompone tareas complejas en subtareas más simples (17), Por ejemplo (”servir la cena”) se descomponen en tareas intermedias (ïr a la cocina”, ”tomar platos”, çolocar en mesa”) y finalmente en acciones primitivas ejecutables por el robot. Este enfoque reduce significativamente la complejidad computacional del problema de planificación al explotar el conocimiento experto sobre cómo descomponer tareas, en contraste con los planificadores clásicos de primer principio que exploran espacios de estados sin guía estructural (18).

3.1.1. Fundamentos de la Planificación Jerárquica: HTN

El método más influyente de planificación jerárquica es el paradigma de **Redes de Tareas Jerárquicas** *(Hierarchical Task Networks, HTN)* (19). En HTN, el problema de planificación no se define mediante un estado objetivo, sino mediante una tarea inicial (task network) que representa el objetivo de alto nivel a alcanzar (17). El conocimiento del dominio se codifica mediante dos elementos fundamentales:

- **Operadores:** Describen acciones primitivas ejecutables directamente por el robot, especificando sus precondiciones y efectos sobre el estado del mundo (20). Son análogos a las acciones en **STRIPS** *(Stanford Research Institute Problem Solver)*, el primer planificador clásico ampliamente adoptado (21).
- **Métodos :** Constituyen el núcleo de la jerarquía. Cada método especifica cómo descomponer una tarea compuesta (abstracta) en una red de subtareas, que pueden ser a su vez compuestas o primitivas (22). Los métodos incluyen precondiciones que determinan cuándo es aplicable una determinada descomposición, permitiendo que el planificador elija entre múltiples formas de realizar una misma tarea según el contexto (23).

El proceso de planificación HTN funciona mediante la descomposición recursiva: partiendo de la tarea inicial, el planificador selecciona métodos aplicables cuyas precondiciones se satisfacen en el estado actual, y reemplaza las tareas compuestas por sus subtareas, generando progresivamente una red completamente expandida en acciones primitivas (**?**). Cuando se alcanza una secuencia de acciones primitivas cuyas precondiciones pueden encadenarse consistentemente desde el estado inicial, se ha encontrado un plan solución.

Una ventaja distintiva de HTN es su capacidad para generar planes explicables y alineados con el conocimiento experto, ya que la jerarquía de métodos refleja cómo un diseñador humano concebiría la realización de tareas complejas (22).

3.1.2. El Planificador HATP (Hierarchical Agent-Based Task Planner)

Dentro de la familia de planificadores HTN orientados a robótica, destaca HATP (Hierarchical Agent-based Task Planner) , desarrollado en LAAS-CNRS para entornos de interacción humano-robot (22, 24). HATP extiende el paradigma HTN tradicional con características específicamente diseñadas para robótica de servicio:

- **Tratamiento de agentes como entidades de primera clase:** HATP distingue explícitamente entre diferentes agentes (robots, humanos) en la representación del dominio, permitiendo asignar tareas a distintos ejecutores y generar planes multiagente sincronizados (25).
- **Reglas sociales (social rules):** Incorpora mecanismos para especificar restricciones sobre comportamientos aceptables en contextos de interacción humano-

robot, como límites de tiempo de espera o secuencias de acciones que deben evitarse por razones de comodidad o seguridad (22).

- **Integración con razonamiento geométrico:** HATP puede intercalar planificación simbólica con validación geométrica mediante planificadores de movimiento (ej. Move3D), verificando en tiempo de planificación si las acciones consideradas son factibles en el mundo físico modelado con detalle 3D (24).

3.1.3. Taxonomía de Arquitecturas Jerárquicas en Robótica

Más allá de HTN, la literatura identifica diversas aproximaciones a la planificación jerárquica en robótica:

- **Planificación Jerárquica con Lógica Temporal (HTL):** Una línea reciente de investigación extiende la planificación jerárquica al dominio de especificaciones de lógica temporal proponiendo un framework de descomposición jerárquica basado en **LTL** *(Linear Temporal Logic)* jerárquico (26), donde cada especificación de alto nivel se descompone en subtareas atómicas, infiriendo relaciones temporales entre ellas para construir una red de tareas. Este enfoque, validado experimentalmente en dominios de navegación y manipulación, permite manejar especificaciones complejas que resultarían inmanejables con una única fórmula LTL monolítica.
- **Arquitecturas de Tres Capas (Three-Layer Architectures):** Un patrón arquitectónico ampliamente adoptado en robótica autónoma es la separación en capas deliberativa, de ejecución y reactiva (27). En el contexto de rehabilitación robótica, esta estructura se manifiesta como:
  - **Capa física:** Maneja la interacción directa con el entorno mediante sensores y actuadores.
  - **Capa reactiva:** Ajusta en tiempo real parámetros de control (ej. asistencia, dificultad) basándose en retroalimentación sensorial.
  - **Capa deliberativa:** Utiliza planificadores como HTN o planificación basada en línea temporal para generar estrategias de intervención personalizadas a largo plazo (28).
- **Planificación Jerárquica con Representaciones Ontológicas:** Investigaciones recientes exploran la integración de ontologías para enriquecer la representación del conocimiento en planificadores jerárquicos (29). Las ontologías permiten modelar relaciones semánticas entre objetos, agentes y tareas, facilitando

el razonamiento sobre contextos no explícitamente programados y mejorando la adaptabilidad del planificador a situaciones novedosas.

3.1.4. Relevancia para el Sistema Propuesto

La arquitectura de planificación jerárquica presentada constituye el fundamento teórico sobre el cual se construye el sistema híbrido CLIPS-ChatGPT/Qwen propuesto en esta tesis. Específicamente:

- El motor CLIPS implementa un sistema de producción que, aunque no sigue estrictamente el formalismo HTN, puede organizarse jerárquicamente mediante reglas de prioridad y descomposición secuencial, como se detalla en la sección 4.1.1 (10).
- La integración con ChatGPT/Qwen para planificación dual (sección 4.3) puede entenderse como una extensión de las arquitecturas híbridas que combinan planificadores simbólicos (como HATP) con módulos de razonamiento geométrico o semántico, pero sustituyendo estos últimos por modelos de lenguaje de gran escala con capacidad de generar planes contextualmente adaptativos (6).
- La arquitectura ViRoot del robot Justina (Figura 3.1) sigue explícitamente una organización jerárquica en capas de entradas, planificación, gestión de conocimiento y ejecución, alineándose con los principios de las arquitecturas de tres capas discutidas (10).

Esta base teórica proporciona el marco necesario para comprender cómo la combinación de razonamiento simbólico jerárquico y procesamiento flexible de lenguaje natural puede superar las limitaciones de ambos paradigmas por separado, constituyendo la hipótesis central de esta investigación.

3.2. Fundamentos de los Sistemas de Producción (Rule-Based Systems)

Los sistemas de producción, también conocidos como sistemas basados en reglas *(Rule-Based Systems, RBS)*, constituyen uno de los paradigmas más establecidos en inteligencia artificial para la representación del conocimiento y el razonamiento automatizado (30). Su fundamento teórico reside en la representación del conocimiento mediante reglas del tipo if-then *(condición-acción)*, que operan sobre una base de he-

chos que describe el estado actual del mundo (31). En robótica, estos sistemas han demostrado particular utilidad para la planificación de acciones en entornos estructurados, donde la previsibilidad y la capacidad de explicación son requisitos fundamentales (30).

3.2.1. Estructura y Componentes de un Sistema de Producción

Un sistema de producción se compone de tres elementos fundamentales que interactúan de manera cíclica (30):

1. **Base de Hechos:** Almacena la información relevante sobre el estado actual del entorno y del propio robot. Estos hechos representan afirmaciones atómicas sobre el mundo, como la ubicación de objetos, el estado de los actuadores o las percepciones sensoriales . En el contexto del robot Justina, la memoria de trabajo contendría hechos como *(ubicacion robot cocina)* o *(objeto vaso mesa)*.
2. **Base de Reglas:** Contiene el conjunto de reglas de producción que codifican el conocimiento del dominio. Cada regla presenta la forma general (31):

$$(condition1)(condition2)...(conditionm) \Rightarrow (action1)(action2)...(actionn)$$

   La parte izquierda *(Left-Hand Side, LHS)* especifica las condiciones que deben satisfacerse para que la regla sea aplicable, mientras que la parte derecha *(Right-Hand Side, RHS)* define las acciones a ejecutar, que típicamente modifican la memoria de trabajo o producen efectos externos (30).
3. **Motor de Inferencia (Inference Engine):** Constituye el mecanismo de control que gobierna la aplicación de las reglas. Es responsable de determinar qué reglas son aplicables en cada momento, seleccionar una entre las posibles, y ejecutar sus acciones (30). El motor de inferencia implementa el ciclo fundamental de operación: el ciclo reconocer-actuar *(recognize-act cycle)* (2).

3.2.2. El Ciclo Reconocer-Actuar (Recognize-Act Cycle)

El ciclo reconocer-actuar constituye el corazón operativo de todo sistema basado en reglas. Este proceso iterativo se desarrolla en las siguientes fases (2) (30):

1. **Reconocimiento:** El motor de inferencia compara el contenido actual de la memoria de trabajo con las condiciones de todas las reglas en la base de conocimiento. Aquellas reglas cuyas condiciones se satisfacen completamente con los hechos existentes se consideran *.ªctivadas"* y pasan a formar parte del conjunto de conflictos (2).
2. **Resolución de Conflictos:** Cuando múltiples reglas resultan aplicables simultáneamente, el motor debe seleccionar una para su ejecución. Esta decisión se realiza mediante estrategias de resolución de conflictos, que pueden basarse en prioridades explícitas de las reglas, especificidad de las condiciones, orden de activación, o políticas específicas del dominio (30).
3. **Actuación:** La regla seleccionada se ejecuta, realizando las acciones especificadas en su parte derecha. Estas acciones típicamente modifican la memoria de trabajo (añadiendo, eliminando o actualizando hechos) y pueden tener efectos secundarios sobre el entorno, y robot (2) (30).
4. **Terminación:** El ciclo se repite hasta que se cumple un criterio de terminación, que puede ser la ausencia de reglas aplicables, la consecución del objetivo deseado, o una condición explícita de parada (30).

3.2.3. El Algoritmo RETE para Emparejamiento Eficiente de Patrones

Una implementación ingenua del ciclo reconocer-actuar comprobaría cada regla contra todos los hechos en cada ciclo, resultando en una complejidad computacional prohibitiva incluso para sistemas de tamaño moderado (32). Para abordar esta limitación, Charles Forgy desarrolló el **algoritmo RETE** durante su doctorado en *Carnegie Mellon University (1979)*, cuya publicación apareció en Artificial Intelligence en 1982. El nombre RETE"proviene del latín y significa red", reflejando la estructura de red que constituye su esencia (32).

Principios Fundamentales del Algoritmo RETE

El algoritmo RETE optimiza el proceso de emparejamiento patrón-objeto mediante dos estrategias clave (32):

1. **Almacenamiento de Estado:** RETE mantiene en memoria los resultados de emparejamientos previos, de modo que solo es necesario procesar los cambios

incrementales en la memoria de trabajo, en lugar de reevaluar todas las reglas desde cero en cada ciclo.

2. **Compartición de subcálculos:** Cuando múltiples reglas comparten condiciones comunes, RETE evalúa dichas condiciones una sola vez y reutiliza los resultados para todas las reglas que las contienen.

Estos principios se implementan mediante la construcción de una red de nodos (RETE network) que representa las condiciones de todas las reglas del sistema (32) (3).

Estructura de la Red RETE

La red RETE se organiza jerárquicamente desde el nodo raíz hasta las hojas que representan reglas completas (32):

- **Nodo Raíz:** Punto de entrada por el que fluyen todos los hechos introducidos en el sistema (3).
- **Nodos Alfa:** Realizan pruebas simples sobre hechos individuales, como comprobaciones de tipo o igualdad de atributos. Cada nodo alfa corresponde a un patrón atómico dentro de las condiciones de las reglas (33). Estos nodos operan de manera independiente y almacenan en su memoria los hechos que satisfacen su prueba (32).
- **Nodos Beta:** Implementan operaciones de unión entre conjuntos de hechos procedentes de diferentes caminos en la red. Cada nodo beta combina los resultados de dos nodos anteriores (típicamente uno alfa y otro beta) verificando que las variables compartidas tengan vinculaciones consistentes (34) (33). La memoria de los nodos beta almacena los pares o tuplas que han superado la condición de unión (32).
- **Nodos Terminales:** Representan reglas completas. Cuando un token alcanza un nodo terminal, significa que se ha encontrado una combinación de hechos que satisface todas las condiciones de la regla correspondiente, quedando esta activada para su ejecución (32) (3).

Funcionamiento Incremental

El proceso de evaluación en RETE es incremental . Cuando se añade, modifica o elimina un hecho en la memoria de trabajo:

1. El hecho ingresa por el nodo raíz y se propaga hacia abajo a través de los nodos alfa cuyas pruebas supera (32).

2. Los nodos alfa actualizan sus memorias y propagan los cambios a los nodos beta conectados.
3. Los nodos beta evalúan las nuevas combinaciones posibles con los tokens almacenados en sus memorias izquierda y derecha, actualizando sus propias memorias y propagando los resultados (34) (3).
4. Finalmente, los cambios alcanzan los nodos terminales, actualizando dinámicamente el conjunto de reglas activadas (32).

Este enfoque incremental reduce drásticamente el trabajo computacional, ya que solo se procesan los cambios efectivos en lugar de reevaluar todo el sistema en cada ciclo. La complejidad temporal del algoritmo RETE es, en promedio, independiente del número total de reglas en el sistema, dependiendo principalmente del número de hechos y de la estructura de las condiciones (35).

3.2.4. Implementación de RETE en CLIPS

CLIPS implementa una versión optimizada del algoritmo RETE adaptada a sus necesidades específicas como sistema de producción de propósito general .

Estructura de Memoria en CLIPS

CLIPS mantiene dos tipos principales de memoria :

- **Memoria de Trabajo:** Almacena los hechos que representan el estado actual del sistema. Cada hecho puede ser simple (ordenado) o estructurado mediante plantillas (deftemplate).
- **Memoria de Hechos:** Una estructura interna que permite el acceso eficiente a los hechos durante el proceso de emparejamiento.

El Motor de Inferencia de CLIPS

El motor de inferencia de CLIPS implementa el ciclo reconocer-actuar completo, utilizando la red RETE para la fase de reconocimiento (36):

1. **Fase de Reconocimiento:** Los cambios en la memoria de trabajo se propagan a través de la red RETE precompilada a partir de las reglas del sistema. La red mantiene información sobre qué condiciones se satisfacen parcial o completamente.
2. **Fase de Resolución de Conflictos:** CLIPS ofrece múltiples estrategias de resolución de conflictos, incluyendo prioridad (salience), profundidad (depth-first),

amplitud (breadth-first), y estrategias basadas en la especificidad de las reglas (36).

3. **Fase de Actuación:** La regla seleccionada ejecuta sus acciones, que pueden incluir la modificación de la memoria de trabajo, la ejecución de funciones externas, o la interacción con el entorno a través de la API de CLIPS.

Funciones Objeto en CLIPS

A partir de CLIPS 6.0, el sistema incorpora capacidades de programación orientada a objetos mediante el módulo COOL (CLIPS Object-Oriented Language) (36). Esta extensión permite:

- Definir clases con atributos y métodos.
- Crear instancias de objetos que residen en una memoria de objetos separada.
- Realizar emparejamiento de patrones sobre objetos mediante la función *object-pattern-match*.

La implementación del algoritmo RETE en CLIPS 6.0 se extendió para soportar el emparejamiento sobre objetos, manteniendo la eficiencia del algoritmo original pero permitiendo representaciones de conocimiento más ricas y estructuradas. Los objetos se tratan como hechos especiales que pueden ser referenciados desde las condiciones de las reglas, y la red RETE maneja tanto hechos tradicionales como objetos de manera unificada (36).

Consideraciones de Rendimiento

Estudios sobre implementaciones de CLIPS en entornos empotrados han identificado cinco parámetros clave que afectan el rendimiento del motor de inferencia :

1. El número de objetos en la memoria de trabajo.
2. La complejidad estructural de los objetos y las reglas.
3. El número de reglas que comparten patrones de emparejamiento comunes.
4. El número de patrones por regla.
5. El número de objetos vinculados a cada patrón.

En aplicaciones de tiempo real el tiempo de respuesta del motor de inferencia puede ser adecuado, aunque los requisitos de memoria pueden ser significativos dependiendo de la complejidad de la red RETE (1). Esta característica hace que CLIPS sea particularmente adecuado para plataformas robóticas con restricciones computacionales moderadas.

3.2.5. Ventajas y Limitaciones de los Sistemas de Producción

Los sistemas basados en reglas presentan ventajas significativas que los hacen útiles en robótica (30):

- **Ventajas:**
  - **Modularidad:** Las reglas pueden añadirse, modificarse o eliminarse independientemente, facilitando el mantenimiento y la evolución del sistema.
  - **Explicabilidad:** El razonamiento puede rastrearse y explicarse mediante la inspección de las reglas activadas, lo que es crucial para la auditoría y la confianza en sistemas críticos.
  - **Naturalidad:** Las reglas expresadas en formato if-then se aproximan a la forma en que los humanos describen procedimientos y conocimientos.
  - **Separación conocimiento-control:** El conocimiento del dominio (reglas) está claramente separado del mecanismo de control (motor de inferencia), permitiendo modificaciones sin afectar la arquitectura subyacente.
- **Limitaciones:**
  - **Dificultad de análisis:** El flujo de control puede ser difícil de predecir cuando múltiples reglas interactúan de maneras complejas.
  - **Ausencia de aprendizaje:** Los sistemas clásicos de producción no incorporan capacidades de aprendizaje automático, requiriendo que todo el conocimiento sea explícitamente programado.
  - **Ineficiencia potencial:** Aunque RETE optimiza el emparejamiento, sistemas con miles de reglas pueden presentar problemas de rendimiento.
  - **Gestión de conflictos:** La resolución de conflictos requiere diseños cuidadosos para evitar comportamientos indeseados.

Estas limitaciones, particularmente la rigidez semántica y la ausencia de aprendizaje, motivan la integración con modelos de lenguaje grande propuesta en esta tesis, buscando combinar la robustez y transparencia de los sistemas basados en reglas con la flexibilidad y capacidad adaptativa de los enfoques subsimbólicos modernos (30).

3.3. Introducción a los Modelos de Lenguaje Grande (LLMs) y ChatGPT/Qwen

Los Modelos de Lenguaje Grande (Large Language Models, LLMs) han emergido como una de las tecnologías más transformadoras en el campo de la inteligencia artificial, revolucionando la manera en que las máquinas procesan y generan lenguaje natural (6). Su aplicación en robótica de servicio abre nuevas posibilidades para la interacción humano-robot, permitiendo que usuarios no expertos comuniquen sus intenciones mediante comandos verbales complejos sin necesidad de conocer lenguajes de programación estructurados (9). Esta sección expone los fundamentos arquitectónicos de los LLMs, centrándose en el mecanismo de atención y la arquitectura Transformer, para posteriormente contrastar dos modelos representativos: ChatGPT (modelo propietario) y Qwen2.5-0.5B (modelo de código abierto), evaluando su idoneidad para entornos robóticos con recursos computacionales limitados.

3.3.1. Fundamentos Arquitectónicos: Transformers y Mecanismo de Atención

Limitaciones de los Modelos Secuenciales Previos

Antes de la introducción de la arquitectura Transformer en 2017, el procesamiento de lenguaje natural estaba dominado por redes neuronales recurrentes (RNN) y sus variantes, como LSTM (Long Short-Term Memory) y GRU (Gated Recurrent Units) (2). Estos modelos procesan el texto de manera secuencial, token por token, lo que presenta dos limitaciones fundamentales (5):

1. **Dificultad para el paralelismo:** La naturaleza secuencial de las RNN impide el aprovechamiento eficiente de hardware paralelo como GPUs, resultando en tiempos de entrenamiento prolongados (33).
2. **Problema de dependencias de largo alcance:** Aunque LSTM y GRU mitigan parcialmente el problema del gradiente evanescente, la información sobre palabras distantes tiende a diluirse a medida que la secuencia se alarga, dificultando el modelado de relaciones semánticas entre elementos lejanos (37).

La Arquitectura Transformer

El artículo seminal .Attention Is All You Need", publicado por investigadores de Google en 2017, introdujo la arquitectura Transformer, que prescinde por completo de recurrencia y convoluciones, basándose únicamente en mecanismos de atención (5). Esta arquitectura se ha convertido en el fundamento de todos los LLMs modernos, incluyendo GPT, BERT, y sus derivados (6).

La arquitectura Transformer sigue una estructura codificador-decodificador (encoder-decoder) con las siguientes características principales (5):

- **Codificador (Encoder):** Compuesto por una pila de $N = 6$ capas idénticas. Cada capa contiene dos subcapas: un mecanismo de autoatención multi-cabeza (multi-head self-attention) y una red feed-forward posicional totalmente conectada. Alrededor de cada subcapa se emplean conexiones residuales seguidas de normalización por capas (layer normalization).
- **Decodificador (Decoder):** Similar al codificador, pero con una tercera subcapa que realiza atención cruzada (cross-attention) sobre la salida del codificador. Además, incorpora un enmascaramiento para evitar que las posiciones futuras influyan en la predicción actual, preservando la propiedad autorregresiva del modelo.

El Mecanismo de Atención

El núcleo de la arquitectura Transformer es el mecanismo de atención, que permite al modelo ponderar dinámicamente la importancia de diferentes elementos de la secuencia de entrada al generar cada representación de salida (5). Formalmente, la atención puede describirse como una función que mapea una consulta (query) y un conjunto de pares clave-valor (key-value) hacia una salida (**?**).

Atención Escalada por Producto Punto (Scaled Dot-Product Attention): La forma de atención utilizada en Transformer se define mediante la siguiente ecuación (5):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{3.1}$$

Interpretación intuitiva del mecanismo de atención: Una analogía útil para comprender este mecanismo es la de una biblioteca (**?**). La biblioteca (Source) contiene muchos libros (Value), cada uno con un código de clasificación (Key). Cuando un usua-

rio desea información sobre un tema específico (Query), el sistema busca los códigos más relevantes y recupera los libros correspondientes, prestando más atención (pesos mayores) a aquellos que mejor coinciden con la consulta.

Autoatención (Self-Attention): Cuando Q, KyV se generan a partir de la misma secuencia de entrada, el mecanismo se denomina autoatención. Esto permite que cada token de la secuencia .ªtienda.ª todos los demás tokens, capturando relaciones contextuales globales independientemente de la distancia entre ellos (5).

Atención Multi-Cabeza (Multi-Head Attention): En lugar de realizar una única función de atención, Transformer proyecta $Q, KyVh$ veces con proyecciones lineales aprendidas diferentes, permitiendo al modelo atender conjuntamente a información procedente de distintos subespacios representacionales (5):

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{3.2}$$

donde $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

Codificación Posicional (Positional Encoding): Dado que la autoatención es inherentemente invariante al orden de los tokens (permutar la secuencia produce el mismo conjunto de atenciones), es necesario inyectar información sobre la posición relativa o absoluta de los tokens. Transformer utiliza codificaciones posicionales sinusoidales (5):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

(3.3)

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

(3.4)

Modelos más recientes han adoptado variantes como RoPE (Rotary Positional Embeddings), que codifica posiciones relativas mediante rotaciones de los vectores QyK, mejorando la extrapolación a longitudes de secuencia superiores a las vistas durante el

entrenamiento (**?**)(**?**).

Evolución hacia Modelos de Lenguaje Grande

A partir de la arquitectura Transformer, la investigación en NLP se bifurcó en dos direcciones principales (**?**):

1. **Modelos tipo codificador (Encoder-only):** Como BERT (Bidirectional Encoder Representations from Transformers), que aprovecha la bidireccionalidad del codificador para tareas de comprensión del lenguaje, como clasificación o reconocimiento de entidades (**?**). Estos modelos se pre-entrenan con objetivos como el modelado de lenguaje enmascarado (Masked Language Modeling, MLM).
2. **Modelos tipo decodificador (Decoder-only):** Como la serie GPT (Generative Pre-trained Transformer), que utilizan únicamente la parte decodificadora con enmascaramiento causal, optimizados para generación de texto autorregresiva (**?**). Estos modelos se pre-entrenan con el objetivo de predicción del siguiente token (Causal Language Modeling, CLM). Los LLMs actuales, incluyendo ChatGPT y Qwen, pertenecen predominantemente a esta categoría.

3.3.2. ChatGPT: Modelo Propietario para Interacción Conversacional

ChatGPT, desarrollado por OpenAI, es un modelo de lenguaje grande basado en la arquitectura GPT (Generative Pre-trained Transformer), específicamente optimizado para tareas de diálogo mediante técnicas de ajuste fino con aprendizaje por refuerzo a partir de retroalimentación humana (RLHF) (6). Su integración en robótica ha demostrado ser prometedora para interpretar comandos complejos en lenguaje natural y generar planes de acción contextualmente apropiados.

- **Características principales de ChatGPT:**
  - **Arquitectura:** Basada en Transformer con decodificador causal, escalada a cientos de miles de millones de parámetros en sus versiones más avanzadas (6). Aunque los detalles arquitectónicos precisos no son públicos, sigue los principios fundamentales establecidos en (5).
  - **Capacidades conversacionales:** Optimizado para mantener diálogos coherentes y contextualmente relevantes, manejar ambigüedades y solicitar aclaraciones cuando es necesario (9).

  - **Acceso mediante API:** OpenAI proporciona interfaces de programación que permiten integrar ChatGPT en sistemas robóticos mediante llamadas a servicios web, facilitando su adopción sin necesidad de infraestructura local costosa (**?**)(**?**).
- **Ventajas para robótica:**
  - **Alta calidad de comprensión:** Demuestra una capacidad superior para interpretar comandos complejos y ambiguos, con tasas de éxito del 100
  - **Flexibilidad semántica:** Puede generar planes adaptativos para situaciones novedosas que no están explícitamente programadas en sistemas basados en reglas (6).
  - **Reducción de latencia:** Investigaciones recientes muestran que la integración directa de ChatGPT con ROS 2 puede reducir la latencia de comunicación en un 7.01 % en promedio, facilitando operaciones robóticas en tiempo real (**?**)(**?**).
- **Limitaciones en entornos robóticos:**
  - **Dependencia de conectividad:** Requiere acceso a internet para consultar la API de OpenAI, lo que puede ser problemático en entornos con conectividad limitada o requisitos de seguridad que impidan la transmisión de datos a la nube (**?**).
  - **Costos operativos:** El uso de API propietarias implica costos por consulta que pueden ser prohibitivos en despliegues a gran escala o con alta frecuencia de interacción (**?**).
  - **Latencia variable:** Los tiempos de respuesta dependen de la calidad de la conexión y la carga de los servidores de OpenAI, lo que puede introducir variabilidad indeseable en sistemas de tiempo real (**?**).
  - **Privacidad de datos:** El envío de comandos de voz y datos contextuales a servidores externos puede plantear problemas de confidencialidad en aplicaciones sensibles (**?**).

3.3.3. Qwen2.5-0.5B: Modelo Open-Source para Cómputo en el Borde

Qwen (QwenLM) es una familia de modelos de lenguaje grande desarrollada por Alibaba, con versiones que abarcan desde modelos masivos hasta variantes extrema-

damente compactas diseñadas para despliegue en entornos con recursos limitados (**?**). Qwen2.5-0.5B-Instruct, la versión con 0.5 mil millones de parámetros, representa un punto óptimo entre capacidad lingüística y eficiencia computacional, siendo particularmente relevante para robótica de servicio embarcada.

- **Características principales de Qwen2.5-0.5B:**

Tabla 3.1: Características principales de Qwen2.5-0.5B-Instruct

Característica	**Descripción**
Parámetros	0.5 mil millones (aproximadamente 1 GB en pesos del modelo) (**?**)
Arquitectura	Transformer con decodificador causal, similar a GPT (**?**)
Soporte de inferencia	CPU nativo, memoria ¡ 2 GB, sin necesidad de GPU (**?**)(**?**)
Idiomas	Optimizado para chino, con capacidades funcionales en inglés (**?**)
Ajuste	Instrucción fine-tuning (versión Instruct) para diálogo y seguimiento de comandos (**?**)
Formato de despliegue	Disponible en GGUF (llama.cpp), ONNX, y formatos nativos de PyTorch (**?**)

- **Ventajas para robótica embarcada:**
  - **Ejecución local y privada:** Todo el procesamiento ocurre en el dispositivo, eliminando dependencias de red y garantizando la privacidad de los datos (**?**).
  - **Bajos requisitos computacionales:** Puede ejecutarse en procesadores ARM (como Raspberry Pi 4/5) y x86 de gama baja, con consumo de memoria inferior a 2 GB tras cuantización INT8 (**?**). Tiempos de inferencia típicos: 300-800 ms para primer token en CPU de 4 núcleos (**?**).
  - **Costo cero por consulta:** Al ser código abierto y ejecutarse localmente, no hay costos asociados al número de interacciones.
  - **Personalización:** Posibilidad de fine-tuning con datos específicos del dominio (por ejemplo, vocabulario de un laboratorio o nombres de objetos domésticos) sin dependencia de proveedores externos (**?**).
  - **Integración con ROS 2:** Experiencias documentadas muestran la viabi-

lidad de integrar Qwen2.5-0.5B con ROS 2 para aplicaciones como guías inteligentes en entornos educativos, donde el modelo procesa consultas en lenguaje natural y genera respuestas que se traducen en comandos de navegación (**?**).

- **Limitaciones:**
  - **Capacidad lingüística reducida:** Con solo 0.5B parámetros, su comprensión y generación son inferiores a modelos como ChatGPT, especialmente para comandos muy complejos o ambiguos (**?**). Sin embargo, para dominios restringidos (vocabulario de un hogar o laboratorio), el rendimiento puede ser suficiente tras personalización.
  - **Requiere optimización:** Para alcanzar latencias aceptables en CPU, es necesario aplicar técnicas de cuantización (INT8, INT4) y utilizar motores de inferencia eficientes como llama.cpp u ONNX Runtime (**?**).
  - **Menor capacidad de diálogo:** La gestión de contextos largos y la coherencia en diálogos extendidos son inferiores a las de modelos con más parámetros (**?**).

3.3.4. Comparativa y Selección para Robótica de Servicio

La Tabla 3.2 resume las principales diferencias entre ChatGPT (representante de modelos propietarios) y Qwen2.5-0.5B (representante de modelos open-source ligeros), proporcionando criterios para la selección según las necesidades de la aplicación robótica.

En el contexto de esta tesis, la selección entre ambos modelos responde a una estrategia complementaria (10):

- **Qwen2.5-0.5B** se emplea como módulo de traducción de lenguaje natural a hechos CLIPS directamente en el robot, aprovechando su capacidad para ejecutarse localmente con latencias predecibles y sin dependencia de conectividad. Esto es particularmente útil para tareas rutinarias donde el vocabulario está acotado al entorno doméstico.
- **ChatGPT** se utiliza en el módulo de planificación dual como generador de planes alternativos para situaciones complejas o novedosas, cuando el sistema basado en reglas no puede resolver la tarea. En estos casos, la latencia adicional se justifica por la necesidad de flexibilidad semántica.

Esta arquitectura híbrida capitaliza las ventajas de ambos paradigmas: la eficiencia y

Tabla 3.2: Comparativa entre ChatGPT y Qwen2.5-0.5B para robótica de servicio

Aspecto	**ChatGPT**	**Qwen2.5-0.5B**
Parámetros	¿100B (estimado)	0.5B
Ejecución	Nube (API)	Local (edge)
Latencia típica	500-2000 ms (variable)	300-800 ms (consistente) (**?**)
Dependencia de red	Obligatoria	Ninguna
Privacidad	Datos enviados a terceros	Datos locales
Costo por consulta	Sí (por token)	No
Personalización	Limitada (fine-tuning no disponible)	Completa (fine-tuning local)
Calidad lingüística	Muy alta	Moderada (suficiente para dominios restringidos) (**?**)
Consumo energético	Alto (servidores)	Bajo (CPU)
Idoneidad	Tareas complejas, investigación, prototipado	Despliegue en producción, dispositivos con recursos limitados, privacidad crítica

privacidad de la ejecución local con Qwen para tareas estándar, y la potencia lingüística de ChatGPT para casos excepcionales, manteniendo siempre la supervisión del sistema basado en reglas como garante de seguridad (6).

3.4. Integración de Traductores de Lenguaje Natural con Sistemas Basados en Reglas

3.4.1. Utilidad y Función de la Integración

3.4.2. Funcionamiento de la Integración CLIPS-Lenguaje Natural

3.4.3. Descripción de Tecnologías: CLIPS, ChatGPT y Qwen2.5-0.5B

3.5. ChatGPT como Agente de Planificación Autónomo

3.5.1. Capacidades de Planificación de LLMs vs. Sistemas Basados en Reglas

Análisis comparativo de los enfoques de planificación, destacando la rigidez predecible de CLIPS frente a la adaptabilidad contextual de los LLMs.

3.5.2. Arquitectura de Planificación Dual: CLIPS y ChatGPT como Motores Complementarios

3.5.3. Mecanismos de Selección y Validación de Planes

3.5.4. Ventajas y Riesgos de la Planificación con LLMs en Robótica

Capítulo 4

Metodología: Diseño del Sistema Híbrido CLIPS-ChatGPT/Qwen

4.1. Diseño del Sistema Basado en Reglas con CLIPS

4.1.1. Definición del Conjunto de Reglas Jerárquicas

Especificación de las reglas organizadas por niveles de prioridad, donde las reglas de seguridad (evitación de colisiones, gestión de emergencias) tienen máxima prioridad sobre las operativas.

4.1.2. Priorización de Acciones Críticas y Gestión de Emergencias

Diseño de reglas específicas para escenarios de fallo y situaciones críticas, asegurando respuestas predecibles y seguras.

4.1.3. Representación del Conocimiento y Hechos en la Base de Datos de CLIPS

Estructuración del conocimiento del entorno doméstico en hechos CLIPS iniciales que sirven como base para el razonamiento.

Descripción de la base de conocimiento inicial incluye:

- *Topología del entorno:* habitaciones, conexiones, zonas navegables.
- *Taxonomía de objetos:* categorías, subcategorías, propiedades heredables.
- *Relaciones espaciales:* contención, adyacencia, orientación.
- *Capacidades del robot:* acciones posibles, restricciones físicas.

4.2. Integración de ChatGPT/Qwen como Sistema Complementario

4.2.1. Funciones Asignadas al Módulo de Lenguaje Natural

4.2.2. Protocolo de Comunicación entre CLIPS y los Modelos de Lenguaje

4.2.3. Mecanismos de Seguridad y Validación para las Respuestas del LLM

Implementación de verificaciones para asegurar que los planes generados por LLMs cumplen con constraints de seguridad y factibilidad robótica.

4.3. Implementación de Planificación Dual

4.3.1. Diseño del Módulo de Planificación con ChatGPT

4.3.2. Protocolos de Prompting para Generación de Planes Robóticos

4.3.3. Mecanismo de Selección entre Planes CLIPS vs. ChatGPT

4.3.4. Validación y Simulación de Planes Antes de Ejecución

Integración con el Robot Justina

- *Procesamiento de Entrada de Voz.*
- *Ejecución de Planes en el Entorno Físico:*

  - Mecanismos para transformar los planes publicados en los tópicos de resultados en comandos ejecutables por los actuadores del robot (navegación, manipulación de objetos, etc.)
- *Retroalimentación y Aprendizaje Incremental.*

Capítulo 5

Implementación

5.1. Entornos de Desarrollo: Simulación y Plataforma Física

5.2. Implementación del Motor de Reglas en CLIPS

5.3. Desarrollo del Módulo de Integración

5.4. Configuración de la Interfaz con API de OpenAI para ChatGPT o Modelo Qwen Local

5.5. Casos de Prueba para Validar la Interacción entre los Módulos

Capítulo 6

Escenarios de Validación y Experimentación

6.1. Diseño de Experimentos en Entornos Controlados

6.1.1. Escenario 1: Ejecución de Tareas Predefinidas (solo CLIPS)

6.1.2. Escenario 2: Gestión de Órdenes Imprecisas o Novedosas (CLIPS + ChatGPT/Qwen)

6.1.3. Escenario 3: Respuesta a Eventos Inesperados o Fallos

6.1.4. Escenario 4: Planificación Dual para Tareas Complejas (CLIPS vs. ChatGPT)

- Comparativa de eficiencia en generación de planes
- Evaluación de robustez ante escenarios novedosos
- Análisis de seguridad en planes generados por LLMs

6.2. Métricas de Evaluación

6.2.1. Métricas Cuantitativas: Tiempo de Ejecución, Tasa de Éxito, Uso de Recursos Computacionales

6.2.2. Métricas Cualitativas: Robustez, Interpretabilidad y Fluidez en la Interacción Humano-Robot

Capítulo 7

Análisis de Resultados

7.1. Comparativa del Rendimiento del Sistema Solo-CLIPS vs. el Sistema Híbrido

7.2. Evaluación de la Efectividad de ChatGPT/Qwen en las Diferentes Funciones Asignadas

7.3. Discusión de Limitaciones y Errores

7.4. Análisis de la Escalabilidad y el Coste Computacional de la Integración

Capítulo 8

Discusión

8.1. Interpretación de los Resultados en el Contexto de los Objetivos

8.2. Ventajas y Desventajas del Enfoque Híbrido Propuesto

8.3. Implicaciones para la Seguridad y Certificación en Entornos Regulados

8.4. Límites Éticos y Prácticos del Uso de ChatGPT/Qwen en Robótica

Capítulo 9

Contribuciones y Relevancia

9.1. Contribución Técnica: Marco Híbrido Escalable y Documentado

9.2. Relevancia Práctica: Aplicaciones en Salud, Industria y Logística

9.3. Relevancia Académica: Benchmark para Sistemas Rule-Based vs. Híbridos

Capítulo 10

Conclusiones y Trabajo Futuro

10.1. Conclusiones Principales

10.2. Propuestas de Trabajo Futuro

- Fine-tuning de un LLM de código abierto para dominio específico
- Mejora de los mecanismos de seguridad y verificación
- Exploración de arquitecturas de integración más profundas

Referencias

[1] R. R. Murphy, *Introduction to AI Robotics*. MIT Press, 2nd ed., 2019.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4th ed., 2020.

[3] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.

[4] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] L. Shi, X. Chen, Y. Hu, J. Li, H. Zhang, and H. Zhao, "Chatgpt for robotics: Design principles and model abilities," *Microsoft Research*, 2023. Preprint.

[7] NASA Johnson Space Center, "Clips reference manual," tech. rep., NASA, 1991.

[8] B. Pell *et al.*, "An autonomous spacecraft agent prototype," in *First International Conference on Autonomous Agents*, pp. 253–261, 1997.

[9] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee, "Robotic pick-and-place with natural language commands," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 4615–4622, 2020.

[10] J. Savage and otros, "Virbot: A modular architecture for service robots with hybrid planning," in *RoboCup Symposium*, 2024.

[11] L. Cheng, Y. Mao, and T. Ward, "Editorial: Neural network models in autonomous robotics," *Frontiers in Neurorobotics*, 2025. En prensa.

[12] R. Raj and A. Kos, "An extensive study of convolutional neural networks: Applications in computer vision for improved robotics perceptions," *Sensors*, 2025. En revisión.

[13] A. Sadek, *Building autonomous agents with hybrid navigation policies*. PhD thesis, Université de Technologie de Compiègne, 2024. HAL Id: tel-xxxxx.

[14] A. Bernabei *et al.*, "Data-driven methods applied to soft robot modeling and control: A review," *IEEE Transactions on Automation Science and Engineering*, 2025. Aceptado.

[15] Z. Zhang *et al.*, "Machine learning applications in parallel robots: A brief review," *Machines*, vol. 13, no. 2, p. 101, 2025.

[16] J. Kim and S. Lee, "A hybrid semantic analysis approach using rule based and learning techniques for human-robot interaction," in *Proceedings of the IEEE International Conference on Human-Robot Interaction*, pp. 345–352, 2024.

[17] D. Nau, T.-C. Au, O. Ilghami, U. Kuter, J. W. Murdock, D. Wu, and F. Yaman, "Shop2: An htn planning system," *Journal of Artificial Intelligence Research*, vol. 20, pp. 379–404, 2003.

[18] K. Erol, J. Hendler, and D. S. Nau, "Htn planning: Complexity and expressivity," in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 1123–1128, 1994.

[19] I. Georgievski and M. Aiello, "An overview of hierarchical task network planning," *Journal of Autonomous Agents and Multi-Agent Systems*, vol. 30, no. 4, pp. 709–752, 2016.

[20] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial Intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.

[21] H. Geffner, "Model-free, model-based, and general intelligence," *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 10–17, 2020.

[22] R. Lallement, L. de Silva, and R. Alami, "Hatp: An htn planner for robotics," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 10–18, 2014. arXiv:1405.5345.

[23] L. de Silva, R. Lallement, and R. Alami, "The hatp hierarchical planner: Formalisation and an initial study of its usability and practicality," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5845–5852, 2016.

[24] L. de Silva and R. Alami, "Dealing with on-line human-robot negotiations in hierarchical agent-based task planner," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 67–75, 2017.

[25] R. Alami, R. Chatila, A. Clodic, *et al.*, "On human-aware task and motion planning," *Robotics and Autonomous Systems*, vol. 61, no. 8, pp. 804–813, 2013.

[26] X. Luo, S. Xu, R. Liu, and C. Liu, "Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6142–6149, 2024.

[27] E. Gat, "On three-layer architectures," in *Artificial Intelligence and Mobile Robots*, pp. 195–210, MIT Press, 1998.

[28] F. Morales *et al.*, "Automated planning and scheduling in robot-aided rehabilitation: A review," *Journal of NeuroEngineering and Rehabilitation*, vol. 22, no. 1, p. 45, 2025.

[29] W. Pang, H. Li, Q. Huang, P. Li, and X. Ma, "Ontology-based mission planning for unmanned systems: A review," *Journal of Unmanned Systems*, vol. 10, no. 2, pp. 112–128, 2022.

[30] Middlesex University, "Recognise act cycle," 2023. Accedido: 2025-03-12.

[31] B. L. Donnell, "Object/rule integration in clips," *Expert Systems*, vol. 11, no. 1, pp. 29–45, 1994.

[32] F. J. Fanning, "An evaluation of an ada implementation of the rete algorithm for embedded flight processors," *Air Force Institute of Technology Theses and Dissertations*, 1990. AFIT-GE-ENG-90D-70.

[33] University of Žilina, "Production rules," 2023. Accedido: 2025-03-12.

[34] ScienceDirect, "Production systems and rules," 2023. Accedido: 2025-03-12.

[35] C. L. Forgy, "Rete: A fast algorithm for the many pattern/many object pattern match problem," *Artificial Intelligence*, vol. 19, no. 1, pp. 17–37, 1982.

[36] C. L. Forgy, *On the efficient implementation of production systems*. PhD thesis, Carnegie-Mellon University, 1979.

[37] L. Albert, "Average case complexity analysis of rete pattern-match algorithm, and average size of join in databases," in *Foundations of Software Technology and Theoretical Computer Science*, vol. 405 of *Lecture Notes in Computer Science*, pp. 223–241, Springer, 1989.